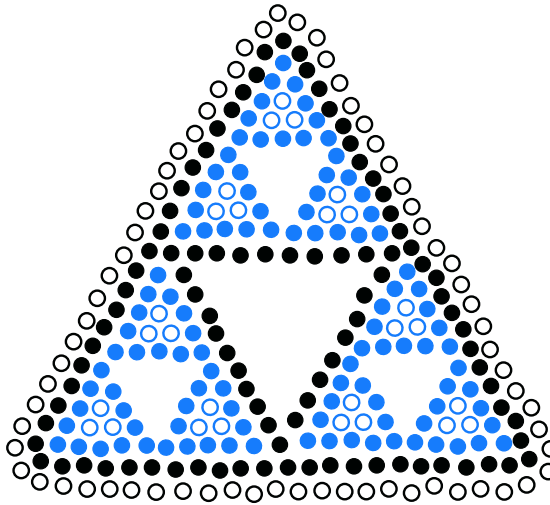




Contents

5.1	Illustrative Examples	191
5.1.1	The Factorial Function	191
5.1.2	Drawing an English Ruler	193
5.1.3	Binary Search	196
5.1.4	File Systems	198
5.2	Analyzing Recursive Algorithms	202
5.3	Further Examples of Recursion	206
5.3.1	Linear Recursion	206
5.3.2	Binary Recursion	211
5.3.3	Multiple Recursion	212
5.4	Designing Recursive Algorithms	214
5.5	Recursion Run Amok	215
5.5.1	Maximum Recursive Depth in Java	218
5.6	Eliminating Tail Recursion	219
5.7	Exercises	221

One way to describe repetition within a computer program is the use of loops, such as Java's while-loop and for-loop constructs described in Section 1.5.2. An entirely different way to achieve repetition is through a process known as *recursion*.

Recursion is a technique by which a method makes one or more calls to itself during execution, or by which a data structure relies upon smaller instances of the very same type of structure in its representation. There are many examples of recursion in art and nature. For example, fractal patterns are naturally recursive. A physical example of recursion used in art is in the Russian Matryoshka dolls. Each doll is either made of solid wood, or is hollow and contains another Matryoshka doll inside it.

In computing, recursion provides an elegant and powerful alternative for performing repetitive tasks. In fact, a few programming languages (e.g., Scheme, Smalltalk) do not explicitly support looping constructs and instead rely directly on recursion to express repetition. Most modern programming languages support functional recursion using the identical mechanism that is used to support traditional forms of method calls. When one invocation of the method makes a recursive call, that invocation is suspended until the recursive call completes.

Recursion is an important technique in the study of data structures and algorithms. We will use it prominently in several later chapters of this book (most notably, Chapters 8 and 12). In this chapter, we begin with the following four illustrative examples of the use of recursion, providing a Java implementation for each.

- The *factorial function* (commonly denoted as $n!$) is a classic mathematical function that has a natural recursive definition.
- An *English ruler* has a recursive pattern that is a simple example of a fractal structure.
- *Binary search* is among the most important computer algorithms. It allows us to efficiently locate a desired value in a data set with upwards of billions of entries.
- The *file system* for a computer has a recursive structure in which directories can be nested arbitrarily deeply within other directories. Recursive algorithms are widely used to explore and manage these file systems.

We then describe how to perform a formal analysis of the running time of a recursive algorithm, and we discuss some potential pitfalls when defining recursions. In the balance of the chapter, we provide many more examples of recursive algorithms, organized to highlight some common forms of design.

5.1 Illustrative Examples

5.1.1 The Factorial Function

To demonstrate the mechanics of recursion, we begin with a simple mathematical example of computing the value of the **factorial function**. The factorial of a positive integer n , denoted $n!$, is defined as the product of the integers from 1 to n . If $n = 0$, then $n!$ is defined as 1 by convention. More formally, for any integer $n \geq 0$,

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1) \cdot (n-2) \cdots 3 \cdot 2 \cdot 1 & \text{if } n \geq 1. \end{cases}$$

For example, $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$. The factorial function is important because it is known to equal the number of ways in which n distinct items can be arranged into a sequence, that is, the number of **permutations** of n items. For example, the three characters a, b, and c can be arranged in $3! = 3 \cdot 2 \cdot 1 = 6$ ways: abc, acb, bac, bca, cab, and cba.

There is a natural recursive definition for the factorial function. To see this, observe that $5! = 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1) = 5 \cdot 4!$. More generally, for a positive integer n , we can define $n!$ to be $n \cdot (n-1)!$. This **recursive definition** can be formalized as

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1)! & \text{if } n \geq 1. \end{cases}$$

This definition is typical of many recursive definitions of functions. First, we have one or more **base cases**, which refer to fixed values of the function. The above definition has one base case stating that $n! = 1$ for $n = 0$. Second, we have one or more **recursive cases**, which define the function in terms of itself. In the above definition, there is one recursive case, which indicates that $n! = n \cdot (n-1)!$ for $n \geq 1$.

A Recursive Implementation of the Factorial Function

Recursion is not just a mathematical notation; we can use recursion to design a Java implementation of the factorial function, as shown in Code Fragment 5.1.

```

1 public static int factorial(int n) throws IllegalArgumentException {
2     if (n < 0)
3         throw new IllegalArgumentException(); // argument must be nonnegative
4     else if (n == 0)
5         return 1; // base case
6     else
7         return n * factorial(n-1); // recursive case
8 }
```

Code Fragment 5.1: A recursive implementation of the factorial function.

This method does not use any explicit loops. Repetition is achieved through repeated recursive invocations of the method. The process is finite because each time the method is invoked, its argument is smaller by one, and when a base case is reached, no further recursive calls are made.

We illustrate the execution of a recursive method using a *recursion trace*. Each entry of the trace corresponds to a recursive call. Each new recursive method call is indicated by a downward arrow to a new invocation. When the method returns, an arrow showing this return is drawn and the return value may be indicated alongside this arrow. An example of such a trace for the factorial function is shown in Figure 5.1.

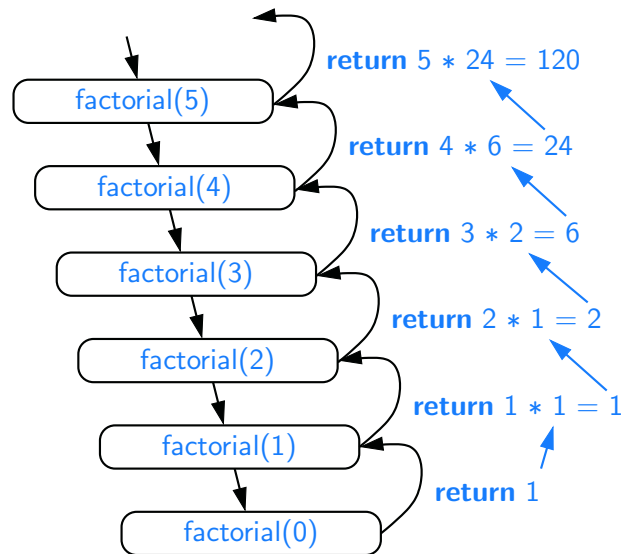


Figure 5.1: A recursion trace for the call `factorial(5)`.

A recursion trace closely mirrors a programming language's execution of the recursion. In Java, each time a method (recursive or otherwise) is called, a structure known as an *activation record* or *activation frame* is created to store information about the progress of that invocation of the method. This frame stores the parameters and local variables specific to a given call of the method, and information about which command in the body of the method is currently executing.

When the execution of a method leads to a nested method call, the execution of the former call is suspended and its frame stores the place in the source code at which the flow of control should continue upon return of the nested call. A new frame is then created for the nested method call. This process is used both in the standard case of one method calling a different method, or in the recursive case where a method invokes itself. The key point is to have a separate frame for each active call.

5.1.2 Drawing an English Ruler

In the case of computing the factorial function, there is no compelling reason for preferring recursion over a direct iteration with a loop. As a more complex example of the use of recursion, consider how to draw the markings of a typical English ruler. For each inch, we place a tick with a numeric label. We denote the length of the tick designating a whole inch as the *major tick length*. Between the marks for whole inches, the ruler contains a series of *minor ticks*, placed at intervals of $1/2$ inch, $1/4$ inch, and so on. As the size of the interval decreases by half, the tick length decreases by one. Figure 5.2 demonstrates several such rulers with varying major tick lengths (although not drawn to scale).

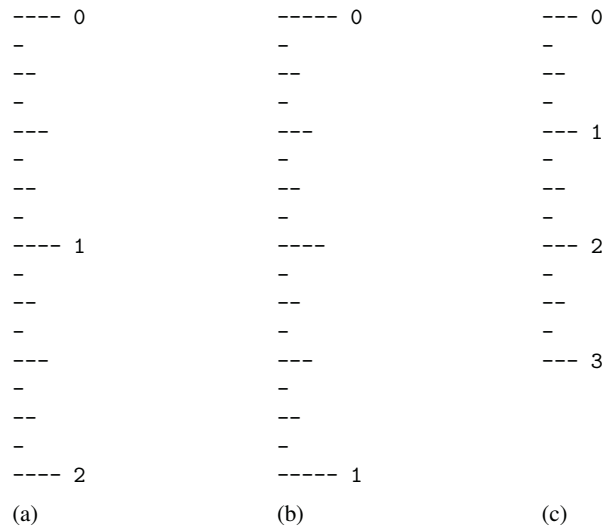


Figure 5.2: Three sample outputs of an English ruler drawing: (a) a 2-inch ruler with major tick length 4; (b) a 1-inch ruler with major tick length 5; (c) a 3-inch ruler with major tick length 3.

A Recursive Approach to Ruler Drawing

The English ruler pattern is a simple example of a *fractal*, that is, a shape that has a self-recursive structure at various levels of magnification. Consider the rule with major tick length 5 shown in Figure 5.2(b). Ignoring the lines containing 0 and 1, let us consider how to draw the sequence of ticks lying between these lines. The central tick (at $1/2$ inch) has length 4. Observe that the two patterns of ticks above and below this central tick are identical, and each has a central tick of length 3.

In general, an interval with a central tick length $L \geq 1$ is composed of:

- An interval with a central tick length $L - 1$
- A single tick of length L
- An interval with a central tick length $L - 1$

Although it is possible to draw such a ruler using an iterative process (see Exercise P-5.29), the task is considerably easier to accomplish with recursion. Our implementation consists of three methods, as shown in Code Fragment 5.2.

The main method, `drawRuler`, manages the construction of the entire ruler. Its arguments specify the total number of inches in the ruler and the major tick length. The utility method, `drawLine`, draws a single tick with a specified number of dashes (and an optional integer label that is printed to the right of the tick).

The interesting work is done by the recursive `drawInterval` method. This method draws the sequence of minor ticks within some interval, based upon the length of the interval's central tick. We rely on the intuition shown at the top of this page, and with a base case when $L = 0$ that draws nothing. For $L \geq 1$, the first and last steps are performed by recursively calling `drawInterval( $L - 1$ )`. The middle step is performed by calling method `drawLine( $L$ )`.

```

1  /** Draws an English ruler for the given number of inches and major tick length. */
2  public static void drawRuler(int nInches, int majorLength) {
3      drawLine(majorLength, 0);           // draw inch 0 line and label
4      for (int j = 1; j <= nInches; j++) {
5          drawInterval(majorLength - 1); // draw interior ticks for inch
6          drawLine(majorLength, j);      // draw inch j line and label
7      }
8  }
9  private static void drawInterval(int centralLength) {
10     if (centralLength >= 1) {           // otherwise, do nothing
11         drawInterval(centralLength - 1); // recursively draw top interval
12         drawLine(centralLength);        // draw center tick line (without label)
13         drawInterval(centralLength - 1); // recursively draw bottom interval
14     }
15 }
16 private static void drawLine(int tickLength, int tickLabel) {
17     for (int j = 0; j < tickLength; j++)
18         System.out.print("-");
19     if (tickLabel >= 0)
20         System.out.print(" " + tickLabel);
21     System.out.print("\n");
22 }
23 /** Draws a line with the given tick length (but no label). */
24 private static void drawLine(int tickLength) {
25     drawLine(tickLength, -1);
26 }

```

Code Fragment 5.2: A recursive implementation of a method that draws a ruler.

Illustrating Ruler Drawing Using a Recursion Trace

The execution of the recursive `drawInterval` method can be visualized using a recursion trace. The trace for `drawInterval` is more complicated than in the factorial example, however, because each instance makes two recursive calls. To illustrate this, we will show the recursion trace in a form that is reminiscent of an outline for a document. See Figure 5.3.

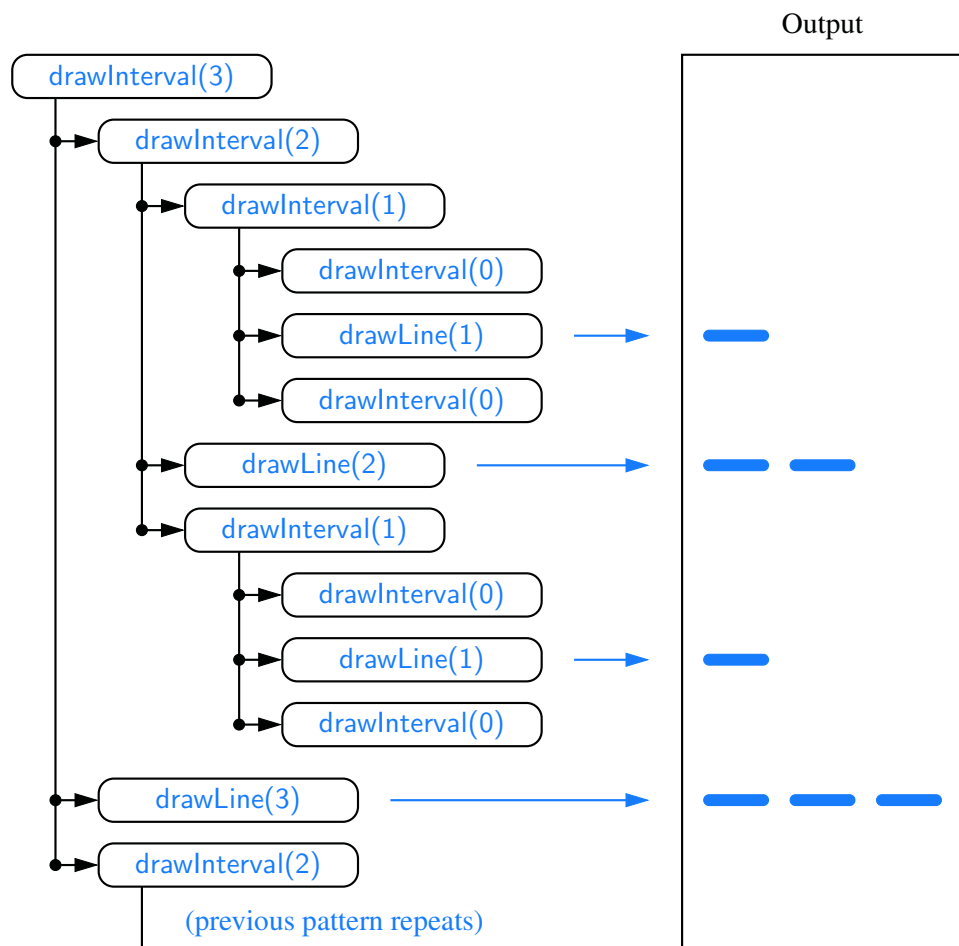


Figure 5.3: A partial recursion trace for the call `drawInterval(3)`. The second pattern of calls for `drawInterval(2)` is not shown, but it is identical to the first.

5.1.3 Binary Search

In this section, we describe a classic recursive algorithm, **binary search**, used to efficiently locate a target value within a sorted sequence of n elements stored in an array. This is among the most important of computer algorithms, and it is the reason that we so often store data in sorted order (as in Figure 5.4).

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
2	4	5	7	8	9	12	14	17	19	22	25	27	28	33	37

Figure 5.4: Values stored in sorted order within an array. The numbers at top are the indices.

When the sequence is **unsorted**, the standard approach to search for a target value is to use a loop to examine every element, until either finding the target or exhausting the data set. This algorithm is known as **linear search**, or **sequential search**, and it runs in $O(n)$ time (i.e., linear time) since every element is inspected in the worst case.

When the sequence is **sorted** and **indexable**, there is a more efficient algorithm. (For intuition, think about how you would accomplish this task by hand!) If we consider an arbitrary element of the sequence with value v , we can be sure that all elements prior to that in the sequence have values less than or equal to v , and that all elements after that element in the sequence have values greater than or equal to v . This observation allows us to quickly “home in” on a search target using a variant of the children’s game “high-low.” We call an element of the sequence a **candidate** if, at the current stage of the search, we cannot rule out that this item matches the target. The algorithm maintains two parameters, low and high, such that all the candidate elements have index at least low and at most high. Initially, low = 0 and high = $n - 1$. We then compare the target value to the **median candidate**, that is, the element with index

$$\text{mid} = \lfloor (\text{low} + \text{high}) / 2 \rfloor.$$

We consider three cases:

- If the target equals the median candidate, then we have found the item we are looking for, and the search terminates successfully.
- If the target is less than the median candidate, then we recur on the first half of the sequence, that is, on the interval of indices from low to mid − 1.
- If the target is greater than the median candidate, then we recur on the second half of the sequence, that is, on the interval of indices from mid + 1 to high.

An unsuccessful search occurs if low > high, as the interval [low, high] is empty.

This algorithm is known as *binary search*. We give a Java implementation in Code Fragment 5.3, and an illustration of the execution of the algorithm in Figure 5.5. Whereas sequential search runs in $O(n)$ time, the more efficient binary search runs in $O(\log n)$ time. This is a significant improvement, given that if n is 1 billion, $\log n$ is only 30. (We defer our formal analysis of binary search's running time to Proposition 5.2 in Section 5.2.)

```

1  /**
2   * Returns true if the target value is found in the indicated portion of the data array.
3   * This search only considers the array portion from data[low] to data[high] inclusive.
4   */
5  public static boolean binarySearch(int[ ] data, int target, int low, int high) {
6      if (low > high)
7          return false;                                // interval empty; no match
8      else {
9          int mid = (low + high) / 2;
10         if (target == data[mid])
11             return true;                              // found a match
12         else if (target < data[mid])
13             return binarySearch(data, target, low, mid - 1); // recur left of the middle
14         else
15             return binarySearch(data, target, mid + 1, high); // recur right of the middle
16     }
17 }

```

Code Fragment 5.3: An implementation of the binary search algorithm on a sorted array.

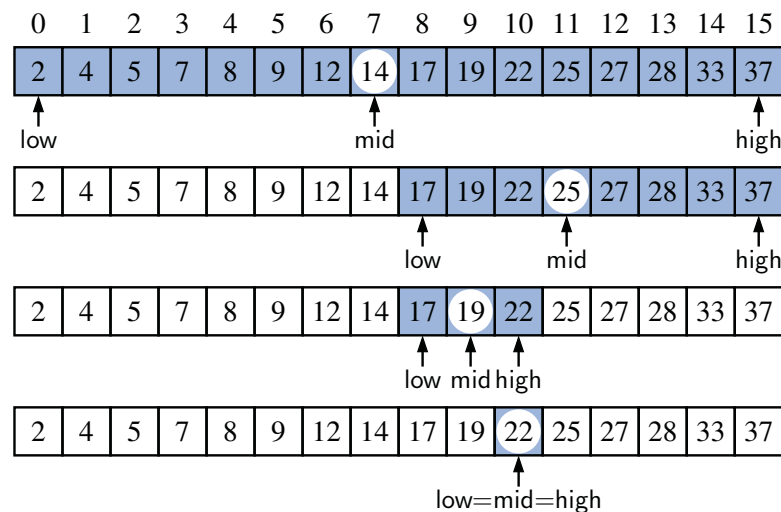


Figure 5.5: Example of a binary search for target value 22 on a sorted array with 16 elements.

5.1.4 File Systems

Modern operating systems define file-system directories (also called “folders”) in a recursive way. Namely, a file system consists of a top-level directory, and the contents of this directory consists of files and other directories, which in turn can contain files and other directories, and so on. The operating system allows directories to be nested arbitrarily deeply (as long as there is enough memory), although by necessity there must be some base directories that contain only files, not further subdirectories. A representation of a portion of such a file system is given in Figure 5.6.

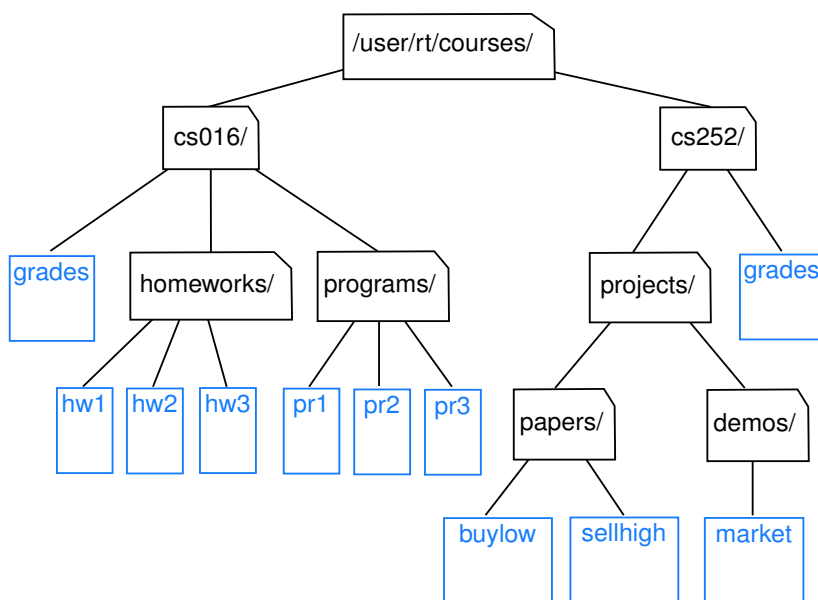


Figure 5.6: A portion of a file system demonstrating a nested organization.

Given the recursive nature of the file-system representation, it should not come as a surprise that many common behaviors of an operating system, such as copying a directory or deleting a directory, are implemented with recursive algorithms. In this section, we consider one such algorithm: computing the total disk usage for all files and directories nested within a particular directory.

For illustration, Figure 5.7 portrays the disk space being used by all entries in our sample file system. We differentiate between the *immediate* disk space used by each entry and the *cumulative* disk space used by that entry and all nested features. For example, the cs016 directory uses only 2K of immediate space, but a total of 249K of cumulative space.

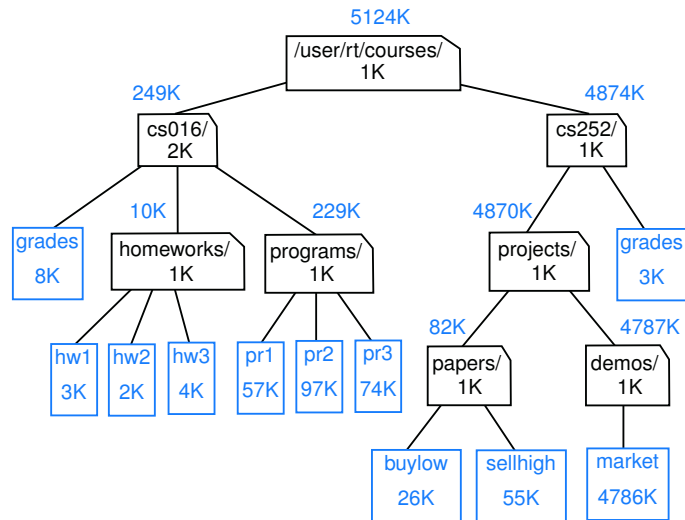


Figure 5.7: The same portion of a file system given in Figure 5.6, but with additional annotations to describe the amount of disk space that is used. Within the icon for each file or directory is the amount of space directly used by that artifact. Above the icon for each directory is an indication of the cumulative disk space used by that directory and all its (recursive) contents.

The cumulative disk space for an entry can be computed with a simple recursive algorithm. It is equal to the immediate disk space used by the entry plus the sum of the cumulative disk space usage of any entries that are stored directly within the entry. For example, the cumulative disk space for `cs016` is 249K because it uses 2K itself, 8K cumulatively in `grades`, 10K cumulatively in `homeworks`, and 229K cumulatively in `programs`. Pseudocode for this algorithm is given in Code Fragment 5.4.

Algorithm `DiskUsage(path)`:

Input: A string designating a path to a file-system entry

Output: The cumulative disk space used by that entry and any nested entries

$total = \text{size}(path)$ {immediate disk space used by the entry}

if $path$ represents a directory **then**

for each $child$ entry stored within directory $path$ **do**

$total = total + \text{DiskUsage}(child)$ {recursive call}

return $total$

Code Fragment 5.4: An algorithm for computing the cumulative disk space usage nested at a file-system entry. We presume that method `size` returns the immediate disk space of an entry.

The java.io.File Class

To implement a recursive algorithm for computing disk usage in Java, we rely on the `java.io.File` class. An instance of this class represents an abstract pathname in the operating system and allows for properties of that operating system entry to be queried. We will rely on the following methods of the class:

- **new File(pathString)** or **new File(parentFile, childString)**
A new `File` instance can be constructed either by providing the full path as a string, or by providing an existing `File` instance that represents a directory and a string that designates the name of a child entry within that directory.
- **file.length()**
Returns the immediate disk usage (measured in bytes) for the operating system entry represented by the `File` instance (e.g., `/user/rt/courses`).
- **file.isDirectory()**
Returns true if the `File` instance represents a directory; false otherwise.
- **file.list()**
Returns an array of strings designating the names of all entries within the given directory. In our sample file system, if we call this method on the `File` associated with path `/user/rt/courses/cs016`, it returns an array with contents: `{"grades", "homeworks", "programs"}`.

Java Implementation

With use of the `File` class, we now convert the algorithm from Code Fragment 5.4 into the Java implementation of Code Fragment 5.5.

```

1  /**
2   * Calculates the total disk usage (in bytes) of the portion of the file system rooted
3   * at the given path, while printing a summary akin to the standard 'du' Unix tool.
4   */
5  public static long diskUsage(File root) {
6      long total = root.length();           // start with direct disk usage
7      if (root.isDirectory()) {            // and if this is a directory,
8          for (String childname : root.list()) { // then for each child
9              File child = new File(root, childname); // compose full path to child
10             total += diskUsage(child); // add child's usage to total
11         }
12     }
13     System.out.println(total + "\t" + root); // descriptive output
14     return total; // return the grand total
15 }

```

Code Fragment 5.5: A recursive method for reporting disk usage of a file system.

Recursion Trace

To produce a different form of a recursion trace, we have included an extraneous print statement within our Java implementation (line 13 of Code Fragment 5.5). The precise format of that output intentionally mirrors the output that is produced by a classic Unix/Linux utility named `du` (for “disk usage”). It reports the amount of disk space used by a directory and all contents nested within, and can produce a verbose report, as given in Figure 5.8.

When executed on the sample file system portrayed in Figure 5.7, our implementation of the `diskUsage` method produces the result given in Figure 5.8. During the execution of the algorithm, exactly one recursive call is made for each entry in the portion of the file system that is considered. Because each line is printed just before returning from a recursive call, the lines of output reflect the order in which the recursive calls are *completed*. Notice that it computes and reports the cumulative disk space for a nested entry before computing and reporting the cumulative disk space for the directory that contains it. For example, the recursive calls regarding entries `grades`, `homeworks`, and `programs` are computed before the cumulative total for the directory `/user/rt/courses/cs016` that contains them.

```
8      /user/rt/courses/cs016/grades
3      /user/rt/courses/cs016/homeworks/hw1
2      /user/rt/courses/cs016/homeworks/hw2
4      /user/rt/courses/cs016/homeworks/hw3
10     /user/rt/courses/cs016/homeworks
57     /user/rt/courses/cs016/programs/pr1
97     /user/rt/courses/cs016/programs/pr2
74     /user/rt/courses/cs016/programs/pr3
229    /user/rt/courses/cs016/programs
249    /user/rt/courses/cs016
26     /user/rt/courses/cs252/projects/papers/buylow
55     /user/rt/courses/cs252/projects/papers/sellhigh
82     /user/rt/courses/cs252/projects/papers
4786   /user/rt/courses/cs252/projects/demos/market
4787   /user/rt/courses/cs252/projects/demos
4870   /user/rt/courses/cs252/projects
3      /user/rt/courses/cs252/grades
4874   /user/rt/courses/cs252
5124   /user/rt/courses/
```

Figure 5.8: A report of the disk usage for the file system shown in Figure 5.7, as generated by our `diskUsage` method from Code Fragment 5.5, or equivalently by the Unix/Linux command `du` with option `-a` (which lists both directories and files).